COMPLETE BUILD ROADMAP — EDGEFORGE SDK

Deploy any LLM to any embedded Linux device in one command.

A universal open-source SDK that solves the hardware fragmentation problem for on-device AI on Raspberry Pi, Orange Pi, industrial boards, and every cheap ARM64 device in between.

12	4	2B+	$0
Month roadmap	Build phases	Potential devices	Required to start

WHY THIS EXISTS

The problem nobody solved

Ollama works on Mac. llama.cpp works if you can compile it. HuggingFace assumes cloud. Every chip vendor ships their own closed SDK. There is no neutral, developer-friendly layer that just works on cheap embedded hardware.

WHAT DEVELOPERS DO TODAY (THE HARD WAY)

```
# Step 1: Install 12 build dependencies and hope nothing conflicts
sudo apt install cmake build-essential git python3-pip libopenblas-dev

# Step 2: Clone and try to compile llama.cpp (45 min on a Pi)
git clone https://github.com/ggerganov/llama.cpp && cd llama.cpp
mkdir build && cd build
```

```
mkdir build && cd build
cmake .. -DGGML_RPI=ON  # pray this flag exists for your board
make -j4  # wait. and wait. thermal throttle. wait more.

# Step 3: Find the right model quantization for your RAM (guess)
wget https://huggingface.co/... # which file? Q4_K_M? Q5_K_S? who knows

# Step 4: Run it and debug why it crashed
FATAL: model requires 4.2GB, only 3.8GB available
# Start over on a different board with different flags
```

This is the real experience. EdgeForge replaces all of that with:

WHAT DEVELOPERS DO WITH EDGEFORGE

```
curl -sSL edgeforge.dev/install | sh

edgeforge run qwen2.5-0.5b
→ Detected: Raspberry Pi 5 (Cortex-A76, 8GB RAM)
→ Selected: qwen2.5-0.5b-Q4_K_M (optimal for your hardware)
→ Binary: pulling precompiled ARM64 build... done
→ Model: downloading 397MB... done
→ Running at 14 tokens/sec. Type to chat.
```

PROBLEM	EDGEFORGE SOLUTION
SoC fragmentation Every board needs different compile flags, backends, and optimizations	**Hardware fingerprinter** Auto-detects board, chip, RAM, and selects the exact right binary and config
45-minute on-device compilation Pi 4 compiling llama.cpp from source is slow, fragile, and fails often	**Precompiled binary registry** Cross-compiled binaries for every target, pulled like Docker images
Model quantization guesswork Developers don't know which GGUF variant fits their RAM envelope	**Hardware-aware model registry** Every model entry carries RAM profiles; SDK picks the right variant automatically

No runtime abstraction

llama.cpp, ONNX Runtime, and vendor NPU SDKs all have different APIs

Unified inference API

One Python/CLI/REST interface regardless of what backend is running underneath

Devices crashing mid-inference

No guard rails on RAM, swap, or thermal limits kills the device under load

Resource governor

Hard limits on RAM, swap, thermal state; graceful degradation to smaller model

SYSTEM DESIGN

Architecture — layer by layer

EdgeForge is five layers. Each layer is independently useful. Together they form the complete platform.

USER SURFACE	**CLI** edgeforge run / list / pull / serve	**Python SDK** from edgeforge import Model	**REST API** OpenAI-compatible /v1/chat/completions
ORCHESTRATION	**Session Manager** Model lifecycle, context, multi-turn	**Resource Governor** RAM limits, thermal, swap guard	**Model Selector** Maps hardware profile → best model variant
HARDWARE	**Fingerprinter** Detects SoC, RAM, cores, NPU, ISA extensions	**Binary Resolver** Pulls right precompiled build for this target	**Delegate Router** CPU / NEON / NPU / OpenCL backend selection
BACKENDS	**llama.cpp** GGUF — primary CPU backend	**ONNX Runtime** Vision & embedding models	**RKNN Toolkit** RK3588 NPU (Orange Pi 5)	**TFLite / LiteRT** Lightweight classification tasks
STRY	**Model Registry**	**Binary Registry**	**Profile DB**

REGI	Models → hardware profiles → download URLs	Precompiled binaries per SoC target	Community-sourced benchmark results per device

Hardware Fingerprinting Logic

The fingerprinter reads from `/proc/cpuinfo`, `/sys/firmware/devicetree/base/model`, `lscpu`, and memory maps to produce a normalized hardware profile.

HARDWARE PROFILE OUTPUT (JSON)

```
{
  "board": "Raspberry Pi 5 Model B",
  "soc": "BCM2712",
  "arch": "aarch64",
  "cores": 4,
  "isa_extensions": ["neon", "fp16", "dotprod", "i8mm"],
  "ram_total_mb": 8192,
  "ram_available_mb": 6800,
  "npu": null,
  "gpu": "VideoCore VII",
  "thermal_zone": "/sys/class/thermal/thermal_zone0",
  "profile_id": "rpi5-bcm2712-8gb-a76"
}
```

Model Registry Schema

Every model entry maps to hardware profiles with explicit RAM requirements, expected performance, and pre-validated download sources.

MODEL REGISTRY ENTRY (YAML)

```
id: qwen2.5-0.5b
name: Qwen 2.5 0.5B
family: qwen
variants:
  - quant: Q4_K_M
    size_mb: 397
    ram_required_mb: 900
    url: https://huggingface.co/...
    sha256: a3f9...
    hardware_profiles:
      - profile: rpi4-bcm2711-4gb
```

```
    profile: rpi4-bcm2711-4gb
    tokens_per_sec: 8
    recommended: true
  - profile: rpi5-bcm2712-8gb-a76
    tokens_per_sec: 14
    recommended: true
  - profile: orangepi5-rk3588-npu
    tokens_per_sec: 31
    backend: rknn
    recommended: true
```

PROJECT LAYOUT

Repository structure

Monorepo with a C core, Python SDK, CLI, and registry. Everything is independently testable.

```
edgeforge/
├── core/                  # C/C++ — hardware detection, binary resolver
│   ├── fingerprint/       # SoC detection, ISA probing
│   ├── governor/          # RAM/thermal/swap resource guard
│   ├── backends/          # llama.cpp, ONNX, RKNN adapters
│   └── CMakeLists.txt
├── sdk/                   # Python SDK (pip install edgeforge)
│   ├── edgeforge/
│   │   ├── model.py       # Model class — load, generate, stream
│   │   ├── hardware.py    # Python wrapper over C fingerprinter
│   │   ├── registry.py    # Pull/cache model and binary registry
│   │   ├── session.py     # Multi-turn conversation management
│   │   └── server.py      # OpenAI-compatible REST server
│   └── pyproject.toml
├── cli/                   # edgeforge CLI (Rust or Python Click)
│   ├── run.py             # edgeforge run <model>
│   ├── pull.py            # edgeforge pull <model>
│   ├── list.py            # edgeforge list
│   ├── serve.py           # edgeforge serve --port 8080
│   └── inspect.py         # edgeforge inspect (show hardware profile)
├── registry/              # JSON/YAML data — models + binaries
│   ├── models.yaml        # All supported models
│   ├── hardware.yaml      # All known hardware profiles
│   └── binaries.yaml      # Precompiled binary download index
├── scripts/               # Cross-compilation scripts (GitHub Actions)
│   ├── build-rpi4.sh
│   ├── build-rpi5.sh
│   ├── build-rk3588.sh
│   └── build-arm64-generic.sh
├── tests/
│   ├── test_fingerprint.py
│   ├── test_registry.py
│   └── test_inference.py
├── docs/                  # Docusaurus site
└── README.md
```

EXECUTION PLAN

Four phases, 12 months

Each phase ships something real and usable. You are never building in private for months. Ship early, get feedback, iterate.

Month 1 Month 4 Month 7 Month 10 Month 12

PHASE 1 **Hardware Foundation — Detect, Download, Run** Month 1 – 3

The goal: one command that works on a Raspberry Pi 4 and Raspberry Pi 5. Nothing more. Prove the core idea.

- Write the hardware fingerprinter in C — reads `/proc/cpuinfo` and `devicetree`, outputs a normalized JSON profile. Test on Pi 4 and Pi 5.
- Build the binary registry — a YAML file listing precompiled llama.cpp builds per target, hosted on GitHub Releases.
- Write the binary resolver — downloads and caches the right prebuilt binary for this device's profile ID.
- Build the model registry — start with 5 models: Qwen2.5-0.5B, Qwen2.5-1.5B, Llama3.2-1B, Phi3-Mini, Gemma-2-2B. Each entry has per-hardware RAM profiles and recommended quant.
- Write the CLI installer script — `curl | sh` that detects arch, pulls binary, sets PATH.
- Write the resource governor — enforces RAM ceiling before model load, watches thermal zone, logs warnings.
- End-to-end test: `edgeforge run qwen2.5-0.5b` works on a cold Pi 4 with zero prior setup. MILESTONE

Week-by-week breakdown

WEEK DELIVERABLE DONE WHEN

WEEK	DELIVERABLE	DONE WHEN
W1–2	Hardware fingerprinter (C) — Pi 4 + Pi 5	Outputs correct JSON profile on both boards
W3	Binary registry schema + 2 precompiled builds	GitHub Release with Pi4 and Pi5 llama.cpp binaries
W4	Binary resolver + model registry (5 models)	Pulls correct binary and model for detected hardware
W5–6	CLI — run, pull, list, inspect commands	All four commands work end-to-end on Pi
W7	Resource governor	Refuses models that exceed available RAM gracefully
W8	Installer script + README	Works on a fresh Raspberry Pi OS install
W9–10	Python SDK layer + OpenAI-compatible REST server	`from edgeforge import Model` works in Python
W11–12	Public launch — GitHub + r/raspberry_pi + HackerNews	Repo live, 100+ GitHub stars SHIP

PHASE 2

Broader Hardware Support + NPU Integration

Month 4 – 6

The goal: support 10+ boards, integrate the RK3588 NPU on Orange Pi 5 (the first real NPU on cheap embedded Linux hardware), and add community-sourced benchmarks.

- → Expand fingerprinter to detect Orange Pi 5 (RK3588), BeagleBone AI-64, Jetson Nano/Orin Nano, Khadas VIM4, Radxa Rock 5B, generic ARM64 embedded Linux.
- → Integrate RKNN Toolkit 2 for the RK3588 NPU — this is the Orange Pi 5's 6 TOPS NPU that almost nothing supports cleanly. This becomes a major differentiator. KEY
- → Build the ONNX Runtime backend adapter for embedding and vision models.
- → Add `edgeforge benchmark` command — measures tokens/sec, RAM usage, peak thermal, tokens/joule for any model on any board.
- → Build the community leaderboard — devices submit benchmark results (opt-in) to a public

HuggingFace dataset. Researchers start citing this data.

- Add `edgeforge serve` — starts an OpenAI-compatible HTTP server on port 8080. Any existing OpenAI client can point to it.
- Write integration tests that run in QEMU for CI/CD — you don't need 10 physical boards to test 10 board profiles.
- Target: 10 boards supported, NPU working on RK3588, benchmark leaderboard live. MILESTONE

PHASE 3 **Developer Experience + Ecosystem** Month 7 – 9

The goal: make it so good that developers choose EdgeForge over manual setup every single time. Documentation, multimodal support, and the Modelfile system.

- Build a Modelfile system (like Ollama's but hardware-aware) — developers define a model configuration with system prompt, RAM limit, and quantization preference, ship it as a single file.
- Add vision model support — MiniCPM-V, moondream2 running on Pi 5 and RK3588. This unlocks camera use cases for robotics and security.
- Add embedding model support — run nomic-embed-text on-device for fully offline vector search applications.
- Build the documentation site (Docusaurus) — Getting Started, Hardware Compatibility Matrix, API Reference, Cookbook with 20 real examples.
- Write a pip package — `pip install edgeforge` installs everything including the C core via pre-built wheels.
- Add RISC-V experimental support — SiFive HiFive Premier and Milk-V Pioneer are the upcoming wave. Being early here is a massive differentiator.
- Submit a technical paper to arXiv describing the hardware abstraction architecture and benchmark methodology. PAPER
- Apply to MLSys 2027 and MobiSys 2027 with the benchmark dataset and architecture paper.

PHASE 4 **Fleet Management + Commercial Tier** Month 10 – 12

The goal: enterprise adoption. Industrial IoT companies, robotics firms, and NGOs deploying

hundreds of devices need management tooling.

- → Build `edgeforge fleet` — a lightweight agent that runs on each device, registers with a central dashboard, and accepts OTA model update commands.
- → Build the fleet dashboard — web UI showing all registered devices, their hardware profiles, running models, and thermal/RAM status in real time.
- → Add model OTA update protocol — push a new model version to 500 Raspberry Pi devices in a factory without SSH-ing into each one.
- → Add hardware-pinned model signing — verify model integrity on the device before loading. Critical for security and medical deployments.
- → Write the commercial license tier — fleet management and OTA are paid features. Everything else stays MIT open source.
- → Reach out to Orange Pi, Radxa, and Khadas for official hardware partnership — they have incentive to bundle EdgeForge as the recommended AI deployment tool for their boards.

IMPLEMENTATION

Technology decisions

Every choice here is deliberate. Wrong language choices kill embedded projects.

Core layer — C/C++

- Hardware fingerprinter runs before Python exists
- Zero dependency on runtime environments
- Direct memory control for resource governor
- Compiles to a 200KB static binary
- Links directly to llama.cpp internals

SDK layer — Python

- Maximum developer reach — everyone has Python
- ctypes/cffi bridge to C core
- FastAPI for the REST server layer
- Distributed via pip with pre-built wheels
- Async-first for streaming inference

CLI — Python (Click) or Rust

- Start with Python Click for speed

Registry — YAML + GitHub

- Model registry is a flat YAML file in the repo

- Rewrite in Rust if startup time matters
- Ships as a standalone binary via PyInstaller
- Consistent UX across ARM32, ARM64, RISC-V

- Community PRs add new hardware profiles
- Binaries hosted on GitHub Releases (free CDN)
- Models pulled from HuggingFace Hub directly
- No backend needed for Phase 1–3

CI/CD — GitHub Actions

- Cross-compile for all targets on every PR
- QEMU for ARM64 integration tests
- Automatic release on version tag push
- Binary signing with checksums on every build

Backends — pluggable adapters

- llama.cpp: primary CPU inference
- ONNX Runtime: vision + embedding models
- RKNN Toolkit 2: RK3588 NPU
- llama.rn principles for mobile crossover
- Each backend is an isolated adapter module

Supported hardware targets — Phase 1 → Phase 4

BOARD	SOC	NPU	PHASE	PRIORITY
Raspberry Pi 4	BCM2711 Cortex-A72	None	Phase 1	Highest — largest install base
Raspberry Pi 5	BCM2712 Cortex-A76	None (RP1 I/O)	Phase 1	Highest — fastest Pi CPU
Orange Pi 5	RK3588 Cortex-A76/A55	6 TOPS RKNN	Phase 2	High — only cheap NPU on Linux
Jetson Nano / Orin Nano	Tegra + CUDA	NVDLA	Phase 2	High — robotics community
BeagleBone AI-64	TDA4VM C7x DSP	MMA DSP	Phase 2	Medium — industrial IoT
Radxa Rock 5B	RK3588S	6 TOPS RKNN	Phase 2	Medium — RK3588 variant
Khadas VIM4	Amlogic A311D2	5 TOPS	Phase 3	Medium
Generic ARM64 Linux	Any ARMv8+	None	Phase 2	High — industrial boards

SiFive HiFive / Milk-V	RISC-V RV64GC	None	Phase 3	Strategic — first mover

ACADEMIC + COMMUNITY STRATEGY

Path to international recognition

You need three things simultaneously: GitHub stars, academic citations, and media coverage. Here is exactly how to get each one.

GitHub and community growth

> **Ship Phase 1 by week 12 and post everywhere on the same day.** r/raspberry_pi (1.2M members), r/LocalLLaMA (350K members), r/selfhosted (400K members), Hacker News Show HN, and the Orange Pi and Raspberry Pi official forums. These communities are actively looking for exactly this. A good Show HN on the right day can get 500 stars in 48 hours.

- Write a benchmarking blog post — "I ran 8 LLMs on a Raspberry Pi 5 and here's what actually works" — technical, honest, with real numbers. This type of post consistently goes viral in embedded communities.
- Build a hardware compatibility matrix page on the docs site that people can contribute to via GitHub PR. Turns users into contributors.
- Make `edgeforge inspect` print a shareable one-liner that people post on Reddit — instant word of mouth.
- Reach out directly to Raspberry Pi Ltd, Orange Pi, and Radxa — they all have official blogs and actively promote software that makes their hardware more useful.

Academic venues

MLSys 2027
Systems paper on hardware abstraction architecture and benchmark methodology

MobiSys 2027
Mobile systems track — on-device inference resource management

arXiv (anytime)
Submit the architecture paper immediately — gets indexed, cited, and found by researchers

IEEE TMC

Transactions on Mobile Computing — benchmark dataset paper

HuggingFace

Publish the benchmark dataset — researchers cite datasets heavily

OSDI / USENIX

Systems track once fleet management layer is mature

The paper angle

The academic contribution is the hardware abstraction architecture itself and the benchmark dataset. Specifically:

- Formal description of the hardware profile schema and how it maps to binary and quantization selection — this is a systems design contribution.
- The benchmark dataset — tokens/sec, RAM, thermal decay curves, and tokens/joule across 10+ boards and 20+ model variants. Nobody has published this at scale. Researchers will cite it constantly.
- Analysis of the RKNN NPU efficiency vs CPU-only inference on RK3588 — first published comparison in this exact context.
- Resource governor design — formal analysis of the RAM/thermal tradeoff space on embedded Linux.

BUSINESS MODEL

How this makes money

The open-source core stays MIT licensed forever. Revenue comes from the layer above it that enterprises need.

Free forever (MIT)

- CLI and Python SDK
- Hardware fingerprinter
- Model registry (community models)
- Single-device inference
- REST server on localhost
- Benchmark tool
- All backends (llama.cpp, ONNX, RKNN)

EdgeForge Pro (paid)

- Fleet management dashboard
- OTA model update protocol
- Hardware-pinned model signing
- HIPAA/GDPR audit logging
- Priority model compilation for custom SoCs
- SLA + enterprise support
- Private model registry hosting

Target buyers

SECTOR	USE CASE	WHY THEY PAY
Industrial IoT	Anomaly detection on factory floor sensors	Need fleet management, audit logs, uptime SLA
Agricultural tech	Offline crop advisory on field devices	No internet in the field, need OTA updates
Healthcare devices	On-device medical triage in rural clinics	HIPAA compliance, no cloud data residency
Security / surveillance	On-device vision reasoning without cloud	Privacy, latency, bandwidth cost
Robotics companies	LLM reasoning on robot compute boards	Reproducible deployment across device fleet
NGOs / Education	Offline AI tutors in low-connectivity schools	Grant-funded, need simple deployment story

> **The realistic trajectory:** 1,000 GitHub stars gets you credibility. 5,000 stars gets you inbound from companies. The fleet management commercial tier targets a $10K–$50K/year enterprise contract. Three contracts = self-sustaining. This is the open-core model that Ollama, Weaviate, and Qdrant all used.

START HERE

What you do on day one

- Create the GitHub repo — `edgeforge/edgeforge`. Write the README first, before any code. Describe the vision clearly. Pin the issue tracker.
- Get a Raspberry Pi 4 or Pi 5 physically in your hands. You will need it for every test. If you don't own one, this is the first investment.
- Read `/proc/cpuinfo` output on a Pi 4 and Pi 5. Write 50 lines of C that parses it and outputs a JSON

- Read `/proc/cpuinfo` output on a Pi 4 and Pi 5. Write 50 lines of C that parses it and outputs a JSON hardware profile. That is the first commit.
- Download a precompiled llama.cpp binary for ARM64. Verify it runs on the Pi manually. Now write code that does exactly this automatically. That is the second commit.
- Write the YAML schema for the model registry. Add Qwen2.5-0.5B as the first entry. That is the third commit.
- Set up GitHub Actions to build and test on every push. This is non-negotiable — CI from day one.
- Do not build the website, the docs site, or the fleet dashboard yet. Phase 1 only. Make the single command work on a real Pi first.

THE ONLY GOAL FOR WEEK 1

```
# On a fresh Raspberry Pi OS, after running your installer:

$ edgeforge run qwen2.5-0.5b

Detected: Raspberry Pi 5 (BCM2712, 8GB)
Pulling binary: llama.cpp-arm64-a76-v0.1.0 ... done
Pulling model: qwen2.5-0.5b-Q4_K_M (397MB) ... done
Running. 14.2 tok/s

> Hello

Hi there! How can I help you today?

# That's it. That's the whole goal for week 1.
```

Everything else — NPU support, fleet management, the paper, the commercial tier — is built on top of this one working moment. Get here first.